Maiml編集API仕様書

作成日: 2026-01-21 16:40
最終更新: 2026-03-06
バージョン: 3.2.0
対象画面: 実験管理者 - Maiml編集画面
総API数: 21個
実装状況: 20個実装済み、1個未実装

📊 実装状況サマリー

カテゴリ	実装済み	未実装	合計
MAIML解析	2	0	2
MAIML操作	4	0	4
バリデーション	2	0	2
セッション管理	2	0	2
ファイル管理	3	1	4
Draft機能	4	0	4
その他	3	0	3
合計	**20**	**1**	**21**

完了率: 95% (20/21)

🎯 API一覧

#	API名	メソッド	エンドポイント	状態	優先度
1	MAIMLパース（基本）	POST	`/api/v1/maiml/parse`	✅ 実装済み	-
2	ペトリネット構造解析（複数メソッド対応）	POST	`/api/v1/maiml/parse-petri-net-structure`	✅ 実装済み	-

3	MAIMLファイル保存	POST	`/api/v1/maiml/save`	✅ 実装済み	-
4	MAIMLファイル読込	POST	`/api/v1/maiml/load`	✅ 実装済み	-
5	MAIMLファイル一覧取得	GET	`/api/v1/maiml/list`	✅ 実装済み	-
6	MAIMLバリデーション	POST	`/api/v1/maiml/validate`	✅ 実装済み	-
7	MAIML自動補正（リアルタイム）	POST	`/api/v1/maiml/auto-correct`	✅ 実装済み	-
8	MAIML自動補正（基本）	POST	`/api/v1/maiml/correct`	✅ 実装済み	-
9	編集セッション開始	POST	`/api/v1/maiml/session/start`	✅ 実装済み	-
10	編集セッション終了	POST	`/api/v1/maiml/session/end`	✅ 実装済み	-
11	ファイル一覧取得	GET	`/api/v1/maiml/files`	✅ 実装済み	-
12	ファイル内容取得	GET	`/api/v1/maiml/file-content`	✅ 実装済み	-
13	ヘルスチェック	GET	`/api/v1/maiml/health`	✅ 実装済み	-
14	Draft保存	POST	`/api/v1/maiml/draft`	✅ 実装済み	-
15	Draft読み込み	GET	`/api/v1/maiml/draft/:draftKey`	✅ 実装済み	-
16	Draft一覧取得	GET	`/api/v1/maiml/drafts`	✅ 実装済み	-
17	Draft削除	DELETE	`/api/v1/maiml/draft/:draftKey`	✅ 実装済み	-
18	名前空間テンプレートフォルダ＆ファイル一覧取得	GET	`/api/v1/maiml/formats`	⏳ 未実装	🔴 高

📋 概要

Maiml編集APIは、MAIMLファイルの読み込み、編集、保存、バリデーション機能を提供します。

主な特徴

1. **複数メソッド対応**: 1つのMAIMLファイルに複数のメソッド（実験手順）を格納
2. **Petri Net構造解析**: XMLからPetri Net構造を抽出
3. **リアルタイムバリデーション**: 編集中のXMLをバリデーション
4. **自動補正機能**: 不正なXMLを可能な範囲で自動修正

使用箇所

MAIMLパースAPIは以下の画面で使用:

1. **プロジェクト管理画面**: ビューアボタン → ViewRendererDialog → ViewRenderer
2. **進捗確認画面**: 詳細ボタン → WorkProgressDetailDialog → PetriNetVisualization経由

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

必須ヘッダー:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

organization_id の取得:

- ログイン時に `X-Organization-Id` ヘッダーから組織IDを取得し、セッションに保存します
- ログイン後のAPIでは、 `Authorization` ヘッダーのセッショントークンから自動的に取得されます
- リクエストヘッダーで `X-Organization-Id` を指定する必要はありません
- `authMiddleware` がセッション情報を検証し、ユーザー情報（organization_idを含む）を `req.user` に格納します

権限要件:

- すべてのAPI: `experiment_admin` または `system_admin` ロール必須

ミドルウェアチェーン:

```
router.METHOD('/path',
  authMiddleware,           // 1. JWT認証（セッション検証、req.userに格納）
  permissionCheck([...]),   // 2. 権限チェック
  controller.handler        // 3. コントローラー処理
);
```

ミドルウェアの動作:

1. `authMiddleware`: JWTトークン（`maiml_session-{sessionId}_{timestamp}`）を検証し、セッション情報を取得して `req.user` に格納（organizationIdを含む）
2. `permissionCheck`: ユーザーのロール（`req.user.role`）を検証
3. コントローラー: `req.user.organizationId` を使用してテナント分離されたデータにアクセス

マルチテナント分離

- すべてのS3パスに `org_{organization_prefix}` プレフィックス必須
- すべてのDBクエリに `organization_id` フィルタ必須

統一レスポンス形式

成功レスポンス:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
```

```
  }
}
```

エラーレスポンス:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

📋 API詳細仕様

1. MAIMLパース（複数メソッド構造取得） 🔴

エンドポイント: `POST /api/v1/maiml/parse`

説明: MAIMLファイルのXMLコンテンツから複数メソッドのPetri Net構造を解析して返却

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

重要: プロジェクト管理画面と進捗確認画面のビューアで使用される主要API

リクエスト

ボディ（JSON）:

```
{
  "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>"
}
```

バリデーション:

フィールド	型	必須	制約
`content`	string	○	XML形式、最大10MB

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "methods": [
      {
        "methodId": "method1",
        "methodName": "合成実験",
        "petriNet": {
          "places": [
            {
              "id": "p1",
              "name": "開始",
              "type": "start",
              "tokens": 1,
              "position": { "x": 100, "y": 100 }
            }
          ],
          "transitions": [
            {
              "id": "t1",
              "name": "材料準備",
              "type": "normal",
              "position": { "x": 250, "y": 100 },
              "templates": {
                "materials": ["material_template_1"],
                "conditions": ["condition_template_1"],
                "results": ["result_template_1"]
              }
            }
          ],
          "arcs": [
```

```
            {
              "id": "arc1",
              "source": "p1",
              "target": "t1",
              "type": "input"
            }
          ]
        }
      },
      {
        "methodId": "method2",
        "methodName": "分析実験",
        "petriNet": { ... }
      }
    ],
    "hasMultipleMethods": true,
    "totalMethods": 2
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: XML形式不正、パース失敗
- 401: 認証エラー
- 403: 権限不足
- 413: ファイルサイズ超過
- 500: システムエラー

実装要件

XMLパース処理:

```
import { parseString } from 'xml2js';

const parseMAIML = (content: string): Promise<any> => {
  return new Promise((resolve, reject) => {
    parseString(content, {
      explicitArray: false,
      ignoreAttrs: false,
      xmlns: false
```

```
    }, (err, result) => {
      if (err) {
        reject(new ValidationError('XML形式が不正です'));
      }
      resolve(result);
    });
  });
};

const maimlData = await parseMAIML(content);
const methods = Array.isArray(maimlData.maiml.methods.method)
  ? maimlData.maiml.methods.method
  : [maimlData.maiml.methods.method];
```

Petri Net構造抽出:

```
const extractPetriNet = (method: any) => {
  const petriNet = method.petriNet;

  return {
    methodId: method.$.id,
    methodName: method.$.name,
    petriNet: {
      places: extractPlaces(petriNet.place),
      transitions: extractTransitions(petriNet.transition),
      arcs: extractArcs(petriNet.arc)
    }
  };
};
```

旧エンドポイントからの移行

旧エンドポイント: `POST /api/worker/multiple-methods-structure-from-content`

非推奨化期間: 2026-01-21 〜 2026-07-21（6ヶ月間）

移行理由:

- RESTful原則に準拠（リソース指向）
- 役割ベースのパス（`/worker/`）を廃止
- 動詞をサブリソースとして配置

---

2. MAIMLファイル保存 🔴

エンドポイント（プロジェクト）: `PUT /api/v1/projects/:projectId/maiml`

エンドポイント（フォーマット）: `PUT /api/v1/formats/:formatId/maiml`

説明: MAIMLファイルをS3に保存（プロジェクトまたはフォーマット）

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectId`	string	○	プロジェクトID（プロジェクトの場合）※2026-02-27: UUID型からvarchar型に変更
`formatId`	UUID	○	フォーマットID（フォーマットの場合）

ボディ（JSON）:

```
{
  "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
  "fileName": "sample_project.maiml",
  "createBackup": true
}
```

バリデーション:

フィールド	型	必須	制約
`content`	string	○	XML形式、最大10MB
`fileName`	string	○	1-255文字、拡張子 `.maiml` 必須
`createBackup`	boolean	×	バックアップ作成フラグ（デフォルト: false）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "file_id": "file-uuid",
    "file_name": "sample_project.maiml",
    "file_path": "org_maiml_lab_001/projects/Sample_Project/sample_project.maiml",
    "file_size": 12345,
    "saved_at": "2026-01-21T16:00:00Z",
    "backup_created": true,
    "backup_path": "org_maiml_lab_001/projects/Sample_Project/archive/sample_project_v2.maiml"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、XML形式不正
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトまたはフォーマットが存在しない
- 413: ファイルサイズ超過
- 500: システムエラー

実装要件

バックアップ処理（プロジェクトの場合）:

```
if (createBackup) {
  // 既存ファイルをarchiveフォルダにコピー
  const archiveFiles = await storageService.listFiles(
    organizationPrefix,
    `projects/${project_name}/archive`
  );

  // 次のバージョン番号を決定
  const nextVersion = archiveFiles.length + 1;
  const backupPath = `projects/${project_name}/archive/${project_name}_v${nextVersion}.maiml`;
```

```
  // 既存ファイルを取得
  const existingContent = await storageService.downloadFile(
    organizationPrefix,
    `projects/${project_name}/${project_name}.maiml`
  );

  // アーカイブに保存
  await storageService.uploadFile(
    organizationPrefix,
    backupPath,
    existingContent,
    { contentType: 'application/xml' }
  );
}
```

XMLバリデーション:

```
// 保存前にXMLバリデーション
try {
  await parseMAIML(content);
} catch (error) {
  throw new ValidationError('XML形式が不正です', error);
}
```

旧エンドポイントからの移行

旧エンドポイント: `POST /api/worker/save-maiml-file`

非推奨化期間: 2026-01-21 〜 2026-07-21（6ヶ月間）

移行理由:

- リソース階層構造を採用（プロジェクト/フォーマット配下のMAIML）
- `context` パラメータ不要（URLで明確に区別）
- PUTメソッド使用（既存リソースの更新、冪等性）

---

3. MAIMLファイル一覧取得 🔴

エンドポイント: `GET /api/v1/maiml-files`

説明: ユーザーがアクセス可能なMAIMLファイル一覧を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`context`	string	×	all	コンテキストフィルタ（`all`, `project`, `format`）
`include_archive`	boolean	×	false	アーカイブファイルを含める

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "projects": [
      {
        "project_id": "project-uuid",
        "project_name": "Sample Project",
        "file_name": "sample_project.maiml",
        "file_path": "org_maiml_lab_001/projects/Sample_Project/sample_project.maiml",
        "file_size": 12345,
        "updated_at": "2026-01-21T16:00:00Z"
      }
    ],
    "formats": [
      {
        "format_id": "format-uuid",
        "format_prefix": "chemistry",
        "file_name": "workflow.maiml",
        "file_path": "org_maiml_lab_001/formats/chemistry/workflow.maiml",
        "file_size": 8901,
        "updated_at": "2026-01-20T15:00:00Z"
      }
    ],
```

```
    "total_count": 15
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 401: 認証エラー
- 403: 権限不足
- 400: バリデーションエラー
- 500: システムエラー

実装要件

データ取得:

```
// プロジェクトのMAIMLファイル
const projectFiles = await db
  .selectFrom('project')
  .where('organization_id', '=', organizationId)
  .select(['project_id', 'project_name'])
  .execute();

// フォーマットのMAIMLファイル
const formatFiles = await db
  .selectFrom('format')
  .where('organization_id', '=', organizationId)
  .where('file_type', '=', 'maiml')
  .select(['format_id', 'format_prefix'])
  .execute();

// S3からファイルメタデータを取得
for (const project of projectFiles) {
  const files = await storageService.listFiles(
    organizationPrefix,
    `projects/${project.project_name}`
  );
  // ファイル情報を付与
}
```

旧エンドポイントからの移行

旧エンドポイント: `GET /api/worker/list-maiml-files`

非推奨化期間: 2026-01-21 〜 2026-07-21（6ヶ月間）

移行理由:

- 複数形のリソース名（`maiml-files`）使用
- kebab-case統一
- 動詞（`list`）を削除

---

4. MAIMLファイル取得 🔴

エンドポイント（プロジェクト）: `GET /api/v1/projects/:projectId/maiml`

エンドポイント（フォーマット）: `GET /api/v1/formats/:formatId/maiml`

説明: 指定されたMAIMLファイルの内容を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectId`	string	○	プロジェクトID（プロジェクトの場合）※2026-02-27: UUID型からvarchar型に変更
`formatId`	UUID	○	フォーマットID（フォーマットの場合）

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`file_name`	string	×	-	特定ファイル名（未指定時はデフォルト）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "file_id": "file-uuid",
    "file_name": "sample_project.maiml",
    "file_path": "org_maiml_lab_001/projects/Sample_Project/sample_project.maiml",
    "file_size": 12345,
    "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
    "loaded_at": "2026-01-21T16:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 404: ファイルが存在しない
- 500: システムエラー

実装要件

ファイルパス解決:

```
let filePath: string;

// プロジェクトの場合
const project = await db
  .selectFrom('project')
  .where('project_id', '=', projectId)
  .where('organization_id', '=', organizationId)
  .select('project_name')
  .executeTakeFirstOrThrow();
```

```
const fileNameToLoad = fileName || `${project.project_name}.maiml`;
filePath = `projects/${project.project_name}/${fileNameToLoad}`;

// S3からファイルを取得
const content = await storageService.downloadFile(
  organizationPrefix,
  filePath
);
```

旧エンドポイントからの移行

旧エンドポイント: `POST /api/worker/load-maiml-file`

非推奨化期間: 2026-01-21 〜 2026-07-21（6ヶ月間）

移行理由:

- GETメソッド使用（リソースの取得、冪等性）
- リソース階層構造を採用
- POSTからGETに変更（適切なHTTPメソッド）

---

5. MAIMLバリデーション 🔴

エンドポイント: `POST /api/v1/maiml/validate`

説明: MAIMLファイルのXMLバリデーションを実行

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエスト

ボディ（JSON）:

```
{
  "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
```

```
  "strict": false
}
```

バリデーション:

フィールド	型	必須	制約
`content`	string	○	XML形式
`strict`	boolean	×	厳密モード（デフォルト: false）

レスポンス

成功時 (200 OK) - バリデーション成功:

```
{
  "success": true,
  "data": {
    "valid": true,
    "errors": [],
    "warnings": [],
    "info": {
      "methodCount": 2,
      "placeCount": 10,
      "transitionCount": 8,
      "arcCount": 18
    }
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

成功時 (200 OK) - バリデーション失敗:

```
{
  "success": true,
  "data": {
    "valid": false,
    "errors": [
      {
        "code": "INVALID_XML_STRUCTURE",
        "message": "閉じタグが不正です",
```

```
        "line": 42,
        "column": 15
      },
      {
        "code": "MISSING_REQUIRED_ELEMENT",
        "message": "methodName属性が必要です",
        "element": "method",
        "line": 10
      }
    ],
    "warnings": [
      {
        "code": "DEPRECATED_ATTRIBUTE",
        "message": "この属性は非推奨です",
        "element": "transition",
        "attribute": "oldAttr",
        "line": 25
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: リクエストボディが不正
- 401: 認証エラー
- 403: 権限不足
- 413: ファイルサイズ超過
- 500: システムエラー

実装要件

バリデーションルール:

```
const validateMAIML = async (content: string, strict: boolean) => {
  const errors: ValidationError[] = [];
  const warnings: ValidationError[] = [];

  // 1. XMLパース
  let maimlData: any;
```

```
  try {
    maimlData = await parseMAIML(content);
  } catch (error) {
    errors.push({
      code: 'INVALID_XML_STRUCTURE',
      message: 'XML形式が不正です',
      details: error.message
    });
    return { valid: false, errors, warnings };
  }

  // 2. 必須要素チェック
  if (!maimlData.maiml.methods) {
    errors.push({
      code: 'MISSING_REQUIRED_ELEMENT',
      message: 'methods要素が必要です'
    });
  }

  // 3. メソッド構造チェック
  const methods = Array.isArray(maimlData.maiml.methods.method)
    ? maimlData.maiml.methods.method
    : [maimlData.maiml.methods.method];

  for (const method of methods) {
    if (!method.$.id || !method.$.name) {
      errors.push({
        code: 'MISSING_REQUIRED_ATTRIBUTE',
        message: 'method要素にはid, name属性が必要です',
        element: 'method'
      });
    }

    // Petri Net構造チェック
    if (!method.petriNet) {
      errors.push({
        code: 'MISSING_REQUIRED_ELEMENT',
        message: 'petriNet要素が必要です',
        element: 'method',
        methodId: method.$.id
      });
    }
  }

  // 4. 厳密モードチェック
  if (strict) {
    // 追加の厳密チェック
    // ...
  }
```

```
  return {
    valid: errors.length === 0,
    errors,
    warnings
  };
};
```

旧エンドポイントからの移行

旧エンドポイント: `POST /api/v1/validation/maiml`

非推奨化期間: 2026-01-21 〜 2026-07-21（6ヶ月間）

移行理由:

- アクション型エンドポイント（API設計ポリシーに準拠）
- 動詞（`validate`）をサブリソースとして配置
- リソース（`maiml`）を主体に

---

6. MAIML自動補正 🔴

エンドポイント: `POST /api/v1/maiml/correct`

説明: 不正なMAIMLファイルを可能な範囲で自動補正

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐ 低

リクエスト

ボディ（JSON）:

```
{
  "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
  "correctionLevel": "safe"
}
```

バリデーション:

フィールド	型	必須	制約
`content`	string	○	XML形式
`correctionLevel`	string	×	`safe`, `moderate`, `aggressive`

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "corrected": true,
    "originalContent": "<?xml version=\"1.0\"...",
    "correctedContent": "<?xml version=\"1.0\"...",
    "corrections": [
      {
        "type": "CLOSE_TAG_ADDED",
        "message": "閉じタグを追加しました",
        "line": 42,
        "element": "transition"
      },
      {
        "type": "ATTRIBUTE_ADDED",
        "message": "必須属性を追加しました",
        "line": 10,
        "element": "method",
        "attribute": "id",
        "value": "method1"
      }
    ],
    "warnings": [
      {
        "message": "一部の要素を削除しました",
        "elements": ["invalidElement1"]
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: 補正不可能
- 401: 認証エラー
- 403: 権限不足
- 413: ファイルサイズ超過
- 500: システムエラー

実装要件

補正レベル:

- **safe**: 明らかな誤りのみ修正（閉じタグ追加、必須属性の補完）
- **moderate**: 構造的な問題も修正（要素の順序、重複削除）
- **aggressive**: 推測による修正も実施（データ型変換、欠落要素の推測補完）

補正処理:

```
const correctMAIML = async (content: string, level: string) => {
  const corrections: Correction[] = [];
  const warnings: Warning[] = [];

  let correctedContent = content;

  // 1. XMLパース試行
  try {
    await parseMAIML(correctedContent);
    return { corrected: false, originalContent: content, correctedContent };
  } catch (error) {
    // パース失敗 → 補正処理へ
  }

  // 2. 基本的な補正（safe以上）
  // 閉じタグの補完
  correctedContent = addClosingTags(correctedContent);
  corrections.push({
    type: 'CLOSE_TAG_ADDED',
    message: '閉じタグを追加しました'
  });

  // 3. 構造的補正（moderate以上）
  if (level === 'moderate' || level === 'aggressive') {
    correctedContent = fixStructure(correctedContent);
  }
```

```
  // 4. 推測補正（aggressive）
  if (level === 'aggressive') {
    correctedContent = inferMissingElements(correctedContent);
  }

  return {
    corrected: true,
    originalContent: content,
    correctedContent,
    corrections,
    warnings
  };
};
```

旧エンドポイントからの移行

旧エンドポイント: `POST /api/v1/correction/maiml`

非推奨化期間: 2026-01-21 〜 2026-07-21（6ヶ月間）

移行理由:

- アクション型エンドポイント（API設計ポリシーに準拠）
- 動詞（`correct`）をサブリソースとして配置
- リソース（`maiml`）を主体に

---

18. 名前空間テンプレートフォルダ＆ファイル一覧取得 🔴

エンドポイント: `GET /api/v1/maiml/formats`

説明: 名前空間テンプレート取り込み機能向けに、全部門横断で `formats/` 配下の `material/` `condition/` `result/` フォルダに存在する `.xml` ファイルの一覧を取得する

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

使用箇所:
`frontend/src/components/editorv2/components/TemplateEditorDialog.tsx`

（名前空間テンプレート取り込みダイアログ）

既存APIが流用不可な理由:

- `GET /api/v1/formats`（4.1）：セッションの `departmentCd` に紐づく単一部門のみ返す
- `GET /api/v1/formats/:folderName/files`（4.2）：同上、全部門横断不可
- `GET /api/v1/maiml/files`（5.11）：`.maiml` / `.xml` 混在、部門フィルタなし

リクエストパラメータ

クエリパラメータなし

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "departments": [
      {
        "departmentCd": "dept_001",
        "departmentName": "化学部門",
        "folders": [
          {
            "folderType": "material",
            "files": [
              {
                "key": "formats/dept_001/material/template1.xml",
                "fileName": "template1.xml",
                "size": 4096,
                "lastModified": "2026-02-01T10:00:00Z"
              },
              {
                "key": "formats/dept_001/material/template2.xml",
                "fileName": "template2.xml",
                "size": 3200,
                "lastModified": "2026-02-10T12:00:00Z"
              }
            ]
          },
          {
            "folderType": "condition",
            "files": [
```

```
            {
              "key": "formats/dept_001/condition/cond_template.xml",
              "fileName": "cond_template.xml",
              "size": 2048,
              "lastModified": "2026-02-05T09:00:00Z"
            }
          ]
        },
        {
          "folderType": "result",
          "files": []
        }
      ]
    },
    {
      "departmentCd": "dept_002",
      "departmentName": "生物部門",
      "folders": [
        {
          "folderType": "material",
          "files": [
            {
              "key": "formats/dept_002/material/bio_material.xml",
              "fileName": "bio_material.xml",
              "size": 5120,
              "lastModified": "2026-02-15T14:00:00Z"
            }
          ]
        },
        {
          "folderType": "condition",
          "files": []
        },
        {
          "folderType": "result",
          "files": []
        }
      ]
    }
  ]
},
"meta": {
  "timestamp": "2026-03-06T10:00:00Z",
  "requestId": "req_abc123"
}
}
```


エラー時:

- 401: 認証エラー
- 403: 権限不足
- 500: システムエラー（S3/Swift接続エラーを含む）

実装要件

処理フロー:

1. セッションから `organizationId` を取得
2. `storageService.listFiles(organizationId, 'formats/')` で全部門フォルダを横断取得
3. S3オブジェクト一覧から部門CD・フォルダ種別・ファイルを分類
4. `material/` ・ `condition/` ・ `result/` 配下の `.xml` ファイルのみ抽出
5. DBの `department` テーブルから `department_name` を取得してレスポンスに付加

実装例（コントローラー）:

```
// backend/src/controllers/maimlController.ts

export const getFormats = async (req: Request, res: Response): Promise<void> => {
  const organizationId = req.user.organizationId;
  const TEMPLATE_FOLDERS = ['material', 'condition', 'result'];

  // S3全オブジェクト取得
  const allFiles = await storageService.listFiles(organizationId, 'formats/');

  // 部門CD → フォルダ種別 → ファイル の構造に分類
  const departmentMap = new Map<string, Map<string, StorageFile[]>>();

  for (const file of allFiles) {
    // key 例: "formats/dept_001/material/template1.xml"
    const parts = file.key.split('/');
    if (parts.length < 4) continue;
    const departmentCd = parts[1];
    const folderType = parts[2];

    if (!TEMPLATE_FOLDERS.includes(folderType)) continue;
    if (!file.key.endsWith('.xml')) continue;

    if (!departmentMap.has(departmentCd)) {
      departmentMap.set(departmentCd, new Map());
    }
```

```
    const folderMap = departmentMap.get(departmentCd)!;
    if (!folderMap.has(folderType)) {
      folderMap.set(folderType, []);
    }
    folderMap.get(folderType)!.push(file);
  }

  // DBから部門名を取得
  const departmentCds = Array.from(departmentMap.keys());
  const departments = await departmentRepository.findByCds(organizationId, departmentCds);
  const deptNameMap = new Map(departments.map(d => [d.department_cd, d.department_name]));

  // レスポンス構造を組み立て
  const result = Array.from(departmentMap.entries()).map(([departmentCd, folderMap]) => ({
    departmentCd,
    departmentName: deptNameMap.get(departmentCd) ?? departmentCd,
    folders: TEMPLATE_FOLDERS.map(folderType => ({
      folderType,
      files: (folderMap.get(folderType) ?? []).map(f => ({
        key: f.key,
        fileName: f.key.split('/').pop(),
        size: f.size,
        lastModified: f.lastModified,
      })),
    })),
  }));

  res.json({ success: true, data: { departments: result } });
};
```

フロントエンドとの連携:

- `templateApi.ts` の `fetchTemplateFiles()` からこのAPIを呼び出す
- ファイル内容取得は既存の `GET /api/v1/maiml/file-content?key=<fileKey>` を流用（`key` にS3フルキーを指定）

マルチテナント分離:

- `storageService.listFiles(organizationId, 'formats/')` により organization_id プレフィックスが自動付与される
- 他組織のファイルへのアクセスは不可

🔒 セキュリティ考慮事項

マルチテナント分離の徹底

S3操作時の必須チェック:

- ☐ すべてのS3操作に `org_{organization_prefix}` プレフィックス
- ☐ クロステナントアクセスの防止

実装例:

```
// ✅ 良い例
const filePath = `projects/${project_name}/${project_name}.maiml`;
const content = await storageService.downloadFile(
  organizationPrefix,  // organization_prefix必須
  filePath
);

// ❌ 悪い例
const content = await storageService.downloadFile(
  '',  // organization_prefixなし
  `projects/${project_name}/${project_name}.maiml`
);
```

ファイルサイズ制限

- **最大ファイルサイズ**: 10MB
- **理由**: メモリ消費とレスポンス時間の制御
- **超過時**: 413 Payload Too Large

XMLインジェクション対策

パース時の設定:

```
parseString(content, {
  explicitArray: false,
  ignoreAttrs: false,
```

```
  xmlns: false,
  strict: true,           // 厳密モード
  normalize: true,        // 正規化
  trim: true,             // トリム
  explicitCharkey: false
}, callback);
```

🔄 旧エンドポイント廃止計画

移行スケジュール

フェーズ	期間	対応内容
Phase 1	2026-01-21 〜 2026-02-20	新エンドポイント実装・テスト
Phase 2	2026-02-21 〜 2026-04-20	並行運用開始、旧エンドポイントに非推奨ヘッダー付与
Phase 3	2026-04-21 〜 2026-06-20	移行促進、警告ログ記録
Phase 4	2026-06-21 〜 2026-07-20	移行完了確認、最終案内
Phase 5	2026-07-21 〜	旧エンドポイント廃止

非推奨化ヘッダー

旧エンドポイントからのレスポンス:

```
HTTP/1.1 200 OK
Deprecation: true
Link: </api/v1/maiml/parse>; rel="successor-version"
Sunset: Wed, 21 Jul 2026 23:59:59 GMT
Warning: 299 - "このエンドポイントは非推奨です。/api/v1/maiml/parse をご使用ください"

{
  "success": true,
  "data": { ... },
  "meta": {
    "deprecationNotice": {
      "message": "このエンドポイントは2026年7月21日に廃止されます",
      "newEndpoint": "/api/v1/maiml/parse",
      "documentationUrl": "https://docs.mmdpf.example.com/api/migration"
    }
```

```
  }
}
```

📚 関連ドキュメント

規約

- API設計ポリシー
- バックエンド規約
- セキュリティ規約

設計書

- API一覧（統合版）
- API実装方針
- S3構造

UI仕様書

- Maiml編集画面

📝 変更履歴

バージョン	日付	変更内容	担当者
3.2.0	2026-03-06	名前空間テンプレートフォルダ＆ファイル一覧取得API追加	Cline
		- API #18: GET /api/v1/maiml/formats 新規追加（未実装）	
		- editorv2 TemplateEditorDialog の名前空間テンプレート取り込み機能向け	
		- 全部門横断で formats/ 配下の material/condition/result の .xml ファイルを返す	

		- レスポンス構造: 部門CD → folderType → files の3階層	
		- 総API数: 20→21個、完了率: 100%→95%	
3.1.0	2026-02-27 15:20	project_id型変更に伴うAPI仕様更新	Cline
		- PUT /api/v1/projects/:projectId/maiml の projectId 型を UUID → string(varchar)に変更	
		- GET /api/v1/projects/:projectId/maiml の projectId 型を UUID → string(varchar)に変更	
3.0.0	2026-02-19 17:04	routeファイルの実装状況を反映	Cline
		- maimlRoutes.tsの全エンドポイントを確認	
		- 実装済み: 20個（100%完了）	
		- 新規追加: Draft機能4エンドポイント、ファイル管理系3エンドポイント	
		- ペトリネット構造解析（複数メソッド対応）追加	
		- 実装ファイル情報を追加	
2.0.0	2026-01-21	RESTful原則に基づきエンドポイント全面改訂	Cline
		- 動詞を含むURLを修正	
		- `/api/worker/` パスを廃止	
		- リソース階層構造を採用	
		- 適切なHTTPメソッドに変更	
		- kebab-case統一	
1.0.0	2024-12-XX	初版作成	Development Team

作成者: Cline
作成日: 2026-01-21 16:40
最終更新: 2026-02-27 15:20